Information Retrieval Final Exam Questions and Answers

Azarbaijan Shahid Madani University - Second Semester of Academic Year 2025-2026

Question 1

Suppose we have a collection of documents with the following characteristics:

- Number of documents (Docs): $1000 \times (Y + 1)$
- Number of words (Vocabulary): $2000 \times (Y + 1)$
- Average length of each word: 7 Byte

Calculate the size of the dictionary in each of the following cases:
1) Dictionary as a string method
2) Using blocking with block size $k = 1$.
(**Note:** Y is the last digit of your student ID number.)

Answer:
Student ID: 4011833230, therefore $Y = 0$.
Number of documents: $1000 \times (0 + 1) = 1000$
Number of words: $2000 \times (0 + 1) = 2000$

Part (a) Dictionary as a string method:
In this method, all characters of the terms are stored sequentially as a single string.

- Total length of the terms string: $2000 \times 7 = 14000$ Byte
- Size of the string pointer: $\log_2(14000) = 13.77 \to 14$ bits $\to 2$ Byte

For each term, we need 4 bytes for frequency and 4 bytes for the pointer to the Posting list. Therefore:

- Space of each term in the dictionary: $4(\text{Freq}) + 4(\text{Post. ptr}) + 2(\text{Str. ptr} + 2) = 10$ Byte
- Total space for the words: $2000 \times 10 = 20000$ Byte
- **Total dictionary space** (including the string): $20000 + 14000 = 34000$ Byte $= 34$ KB

Part (b) Using blocking with block size $k = 1$:

- Number of blocks: $\frac{2000}{1} = 2000$
- Space for frequency and list pointers: $2000 \times (4 + 4) = 16000$ Byte
- Space for terms (string of words): $2000 \times 7 = 14000$ Byte

- Space for Term length: $2000 \times 1 = 2000$ Byte
- Space for block pointers: $2000 \times 2 = 4000$ Byte
- **Total dictionary space:** $16000 + 14000 + 2000 + 4000 = 36000$ Byte $= 36$ KB

Amount of savings: $34000 - 36000 = -2000$ Byte
(In this case, no savings are made, and 2000 more bytes are consumed.)

Question 2

Consider the following set of numbers as the Posting list for word T:

$$X, 100, Y, 300$$

(X is equal to the sum of the last two digits of your student ID, and $Y = 150 + Z$, where Z is equal to the product of the last two digits of your student ID.)
By applying the Variable Byte compression algorithm, calculate the size of the compressed Posting list. For encoding, calculate the Gap and encode it.

Answer:

The last digits of the student ID are 3 and 0, therefore:

$$X = 3 + 0 = 3, \qquad Z = 3 \times 0 = 0, \qquad Y = 150 + 0 = 150.$$

Standard posting list: $3, 100, 150, 300$
Calculating the Gaps:

- $Gap_1 = 3$
- $Gap_2 = 100 - 3 = 97$
- $Gap_3 = 150 - 100 = 50$
- $Gap_4 = 300 - 150 = 150$

List of gaps: $\{3, 97, 50, 150\}$
Encoding phase (Variable Byte):

- Number 3 is less than 128 $\rightarrow$ needs one byte: `10000011`
- Number 97 is less than 128 $\rightarrow$ needs one byte: `11100001`
- Number 50 is less than 128 $\rightarrow$ needs one byte: `10110010`
- Number 150 is greater than 127 $\rightarrow$ $150 = 1 \times 128 + 22$
  The binary equivalent of 1 is `0000001` and the binary equivalent of 22 is `0010110` $\rightarrow$ needs two bytes: `10010110 00000001` (continuation byte and final byte)

Total size of the compressed Posting list: $1 + 1 + 1 + 2 = 5$ Byte.

Question 3

Consider the word T. Its Posting list is as follows:

T: 10, 11, 55, 261

Calculate the size of the compressed Posting list by applying the Gamma code.

Answer:

First, we calculate the Gaps:

- $Gap_1 = 10$
- $Gap_2 = 11 - 10 = 1$
- $Gap_3 = 55 - 11 = 44$
- $Gap_4 = 261 - 55 = 206$

List of gaps: $\{10, 1, 44, 206\}$

Calculating the Gamma code for each Gap: Total compressed size: $7 + 1 +$

Gap	Binary	Length (Unary)	Offset	Gamma Code
10	1010	1110	010	1110010
1	1	0	-	0
44	101100	111110	01100	11111001100
206	11001110	11111110	1001110	111111101001110

$11 + 15 = 34$ bits.

Question 4

What is the most important drawback of the vector space model?

Answer:

The first issue is that it assumes terms are independent and does not understand the semantic relationships between them. The second issue is that it ignores the composition and positional order of words in a document; for this reason, two documents that have exactly the same words, but are completely different semantically, might have the identical vector.

Question 5

What is the most important bottleneck in a ranked retrieval system? What solution do you propose to solve it?

Answer:

In these systems, the main bottleneck is calculating the document scores. This is because the similarity of all documents to the user's query must be calculated, which becomes a significant and costly challenge when the number of documents is large.

To overcome this challenge, **Pruning** methods can be proposed. For example, one could consider only a set A of candidate documents that are likely winners, and reduce the problem to a selection problem, such that $K < |A| \ll N$ (where N is the total number of documents). Furthermore, other pruning methods such as Cluster-based pruning are also available to improve performance.

Question 6

Consider the following term-document matrix. Given the query q = classification algorithms and using the vector space model and the tf-idf weighting method, find the similarity of the query with documents d_2 and d_3 by calculating the cosine similarity. (Use raw tf without taking the logarithm.)

	d_1	d_2	d_3	d_4	d_5
learning	0	0	0	1	1
clustering	1	1	1	0	0
regression	1	0	1	0	0
classification	0	1	1	0	0
precision	0	1	0	0	1
algorithms	0	0	0	1	1

Answer:

The total number of documents is $N = 5$.

Weighting formulas: $idf_t = \log_{10}\left(\frac{N}{df_t}\right)$ and $w_{t,d} = tf_{t,d} \times idf_t$.

Calculating df and idf for the words:

- $df_{\text{learning}} = 2$
- $df_{\text{clustering}} = 3 \rightarrow idf = \log_{10}(5/3) \approx 0.222$
- $df_{\text{regression}} = 2 \rightarrow idf = \log_{10}(5/2) \approx 0.398$
- $df_{\text{classification}} = 2 \rightarrow idf = \log_{10}(5/2) \approx 0.398$
- $df_{\text{precision}} = 2 \rightarrow idf = \log_{10}(5/2) \approx 0.398$
- $df_{\text{algorithms}} = 2 \rightarrow idf = \log_{10}(5/2) \approx 0.398$

Query vector q = classification algorithms:

According to the order of words in the matrix, the query vector is:

$\vec{q} = (0, 0, 0, 0.398, 0, 0.398)$

$||\vec{q}|| = \sqrt{0.398^2 + 0.398^2} \approx 0.563$

Calculating similarity with document d_2:

Document d_2 contains the words clustering, classification, precision.

$\vec{d_2} = (0, 0.222, 0, 0.398, 0.398, 0)$

$||\vec{d_2}|| = \sqrt{0.222^2 + 0.398^2 + 0.398^2} \approx 0.605$

$\text{Cos}(q, d_2) = \frac{\vec{q} \cdot \vec{d_2}}{||\vec{q}|| \times ||\vec{d_2}||} = \frac{0.398 \times 0.398}{0.563 \times 0.605} = \frac{0.1584}{0.3406} \approx 0.465$

Calculating similarity with document d_3:

Document d_3 in the question text contains clustering, regression. (In the student's handwritten solution, it was mistakenly considered to also include classification, which led to equal similarities. The student's solution is provided here):

$\vec{d_3} = (0, 0.222, 0.398, 0.398, 0, 0)$

$||\vec{d_3}|| = \sqrt{0.222^2 + 0.398^2 + 0.398^2} \approx 0.605$

$\text{Cos}(q, d_3) = \frac{\vec{q} \cdot \vec{d_3}}{||\vec{q}|| \times ||\vec{d_3}||} = \frac{0.398 \times 0.398}{0.563 \times 0.605} \approx 0.465$

Therefore, the similarities of d_2 and d_3 with the query were calculated to be equal in the student's solution.

Question 7 (Optional)

Suppose an information retrieval system is given with the following dictionary:

1: clean 2: colon 3: color 4: colour 5: column

What steps must be taken to answer the query `col*` using the Permuterm index? Write your answer accurately.

Answer:

Query: `col*`

First, we add the end-of-string character (`$`) to the end of the query: `col*$`

Then, we perform a circular shift until the `*` character reaches the end of the string: `$col*`

Now we check which dictionary words match the prefix `$col` (by adding `$` to the end and generating permutations):

1. **Word `clean`**: `clean$` → `$clean, n$clea, an$cle, ean$cl, lean$c`
   (None match `$col`)

2. **Word `colon`**: `colon$` → `$colon, n$colo, on$col, lon$co, olon$c`
   (The permutation `$colon` matches `$col` → **It is an answer**)

3. **Word `color`**: `color$` → `$color, r$colo, or$col, lor$co, olor$c`
   (The permutation `$color` matches `$col` → **It is an answer**)

4. **Word `colour`**: `colour$` → `$colour, r$colou, ur$colo, our$col, lour$co, olour$c`
   (The permutation `$colour` matches `$col` → **It is an answer**)

5. **Word `column`**: `column$` $\rightarrow$ `$column, n$colum, mn$colu, umn$col, lumn$co, olumn$c` (The permutation `$column` matches `$col` $\rightarrow$ **It is an answer**)

Final result: The words corresponding to IDs 2, 3, 4, and 5 are matches.